

Grading

The final score that you will receive for your programming assignment is generated in relation to the total points set in your programming assignment item—not the total point value in the nbgrader notebook.

When calculating the final score shown to learners, the programming assignment takes the percentage of earned points vs. the total points provided by nbgrader and returns a score matching the equivalent percentage of the point value for the programming assignment.

DO NOT CHANGE VARIABLE OR METHOD SIGNATURES The autograder will not work properly if you change the variable or method signatures.

Ensemble methods (adaBoost, random forests)

```
In [20]: import numpy as np
import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn.base import clone
from sklearn import tree
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
%matplotlib inline
```

Run the helper code below to create a training and validation set of threes and eights from the MNIST dataset. There is also a helper function to display digits.

```
In [21]: class ThreesandEights:
        """
        Class to store MNIST 3s and 8s data
        """

        def __init__(self, location):

            import pickle, gzip

            # Load the dataset
            f = gzip.open(location, 'rb')

            # Split the data set
            x_train, y_train, x_test, y_test = pickle.load(f)

            # Extract only 3's and 8's for training set
            self.x_train = x_train[np.logical_or(y_train== 3, y_train == 8), :]
            self.y_train = y_train[np.logical_or(y_train== 3, y_train == 8)]
            self.y_train = np.array([1 if y == 8 else -1 for y in self.y_train])
```

```

        # Shuffle the training data
        shuff = np.arange(self.x_train.shape[0])
        np.random.shuffle(shuff)
        self.x_train = self.x_train[shuff,:]
        self.y_train = self.y_train[shuff]

        # Extract only 3's and 8's for validation set
        self.x_test = x_test[np.logical_or(y_test== 3, y_test == 8), :]
        self.y_test = y_test[np.logical_or(y_test== 3, y_test == 8)]
        self.y_test = np.array([1 if y == 8 else -1 for y in self.y_test])

    f.close()

def view_digit(ex, label=None, feature=None):
    """
    function to plot digit examples
    """
    if label: print("true label: {:d}".format(label))
    img = ex.reshape(21,21)
    col = np.dstack((img, img, img))
    if feature is not None: col[feature[0]//21, feature[0]%21, :] = [1, 0, 0]
    plt.imshow(col)
    plt.xticks([]), plt.yticks([])

data = ThreesandEights("data/mnist21x21_3789.pklz")

```

Problem 1: Building an Adaboost Classifier to classify MNIST digits 3 and 8.

Recall that the model we attempt to learn in AdaBoost is given by

$$H(\mathbf{x}) = \text{sign} \left[\sum_{k=1}^K \alpha_k h_k(\mathbf{x}) \right]$$

where $h_k(\mathbf{x})$ is the k^{th} weak learner and α_k is it's associated ensemble coefficient.

```

In [22]: class AdaBoost:
        def __init__(self, n_learners=20, base=DecisionTreeClassifier(max_depth=
        """
        Create a new adaboost classifier.

        Args:
            N (int, optional): Number of weak learners in classifier.
            base (BaseEstimator, optional): Your general weak learner
            random_state (int, optional): set random generator. needed for

        Attributes:
            base (estimator): Your general weak learner
            n_learners (int): Number of weak learners in classifier.
            alpha (ndarray): Coefficients on weak learners.
            learners (list): List of weak learner instances.
        """

```

```

np.random.seed(random_state)

self.n_learners = n_learners
self.base = base
self.alpha = np.zeros(self.n_learners)
self.learners = []

def fit(self, X_train, y_train):
    """
    Train AdaBoost classifier on data. Sets alphas and learners.

    Args:
        X_train (ndarray): [n_samples x n_features] ndarray of training
        y_train (ndarray): [n_samples] ndarray of data
    """

    # =====
    # TODO

    # Note: You can create and train a new instantiation
    # of your sklearn decision tree as follows
    # you don't have to use sklearn's fit function,
    # but it is probably the easiest way

    # w = np.ones(len(y_train))
    # h = clone(self.base)
    # h.fit(X_train, y_train, sample_weight=w)

    # =====

    # your code here

    n = len(y_train)

    # initialize sample weights equally
    w = np.ones(n) / n

    for k in range(self.n_learners):
        h = clone(self.base) # clone base learner to get fresh model
        h.fit(X_train, y_train, sample_weight=w) # fit learner with curr
        y_pred = h.predict(X_train) # get predictions on training model

        # compute weighted classification error
        err = self.error_rate(y_train, y_pred, w)

        # avoid division by 0 or log(0)
        err = np.clip(err, 1e-10, 1 - 1e-10)

        # compute alpha (learner weight) using AdaBoost formula
        alpha_k = 0.5 * np.log((1 - err) / err)

        # save learner and alpha
        self.learners.append(h)
        self.alpha[k] = alpha_k

    # update weights

```

```

        w = w * np.exp(-alpha_k * y_train * y_train)
        w /= np.sum(w)
    return self

def error_rate(self, y_true, y_pred, weights):
    # =====
    # TODO

    # Implement the weighted error rate
    # =====
    # your code here
    return np.sum(weights * (y_true != y_pred)) / np.sum(weights)

def predict(self, X):
    """
    Adaboost prediction for new data X.

    Args:
        X (ndarray): [n_samples x n_features] ndarray of data

    Returns:
        yhat (ndarray): [n_samples] ndarray of predicted labels {-1,1}
    """

    # =====
    # TODO
    # =====

    yhat = np.zeros(X.shape[0])

    # your code here
    # accumulate weighted predictions from each learner
    for alpha_k, h in zip(self.alpha, self.learners):
        yhat += alpha_k * h.predict(X)

    # final prediction is the sign of the weighted sum
    return np.sign(yhat)

def score(self, X, y):
    """
    Computes prediction accuracy of classifier.

    Args:
        X (ndarray): [n_samples x n_features] ndarray of data
        y (ndarray): [n_samples] ndarray of true labels

    Returns:
        Prediction accuracy (between 0.0 and 1.0).
    """

    # your code here
    yhat = self.predict(X)
    return np.mean(yhat == y)

def staged_score(self, X, y):
    """
    Computes the ensemble score after each iteration of boosting

```

for monitoring purposes, such as to determine the score on a test set after each boost.

Args:

X (ndarray): [n_samples x n_features] ndarray of data
y (ndarray): [n_samples] ndarray of true labels

Returns:

scores (ndarray): [n_learners] ndarray of scores
.....

```
scores = []
```

```
# your code here
```

```
yhat = np.zeros(X.shape[0]) # running prediction sum
```

```
for k in range(self.n_learners):
```

```
    # add current learner's weighted prediction
```

```
    yhat += self.alpha[k] * self.learners[k].predict(X)
```

```
    # evaluate and store accuracy so far
```

```
    scores.append(np.mean(np.sign(yhat) == y))
```

```
return np.array(scores)
```

1.a: [25 pts] In the `AdaBoost` class above, implement the `fit` method to learn the sequence of weak learners $\{h_k(\mathbf{x})\}_{k=1}^K$ and corresponding coefficients $\{\alpha_k\}_{k=1}^K$. Note that you may use sklearn's implementation of `DecisionTreeClassifier` as your weak learner which allows you to pass as an optional parameter the weights associated with each training example. An example of instantiating and training a single learner is given in the comments of the `fit` method.

Recall that the AdaBoost algorithm is as follows:

for k=1 to K:

a) Fit kth weak learner to training data with weights w

b) Computed weighted error err_k for the kth weak learner

c) compute vote weight $\alpha[k] = 0.5 \ln((1 - err_k) / err_k)$

d) update training example weights $w[i] \leftarrow \exp(-\alpha[k] y[i] h[k](x[i]))$

e) normalize training weights so they sum to 1

1.b [5 pts] Use the fit function to fit the Adaboost classifier with 150 base decision tree stumps.

```
In [23]: # use fit function to fit Adaboost classifier called clf with 150 base decis
# your code here

# create decision stump
base_stump = DecisionTreeClassifier(max_depth=1)

clf = AdaBoost(n_learners=150, base=base_stump)

clf.fit(data.x_train, data.y_train)
```

```
Out[23]: <__main__.AdaBoost at 0x16043dd30>
```

```
In [24]: # tests using the fit function to fit AdaBoost classifier with 150 base deci
```

1.c [10 pts]: After your `fit` method is working properly, implement the `predict` method to make predictions for unseen examples. You can test out the predictions in the cell below. **Note:** Remember that AdaBoost assumes that your predictions are of the form $y \in \{-1, 1\}$.

Just print out your predictions on the training set in the cell below.

```
In [25]: # print out predictions on the training set
# your code here

train_predict = clf.predict(data.x_train)
print(train_predict)
```

```
[-1. -1.  1. ... -1. -1.  1.]
```

```
In [26]: print(max(train_predict))
```

```
1.0
```

```
In [27]: # tests train_predict which uses the predict method
```

1.d [15 pts]: Once your `predict` function is written up, you need to test the scores on the function. To do this compute the scores on the prediction in the `score` function. Use the `score` function to then complete `staged_score` to collect the scores for every boosting iterations. Plot the misclassification error for train and test sets (misclassification error = 1- score).

```
In [28]: # plot misclassification error for train and test sets
# your code here

def score(self, X, y):
    """
    Computes prediction accuracy classifier
    """
    yhat = self.predict(X)
    return np.mean(yhat == y)

def staged_score(self, X, y):
```

```

'''
Computes ensemble score after each iteration
Returns accuracy at each stage
'''

scores = []
yhat = np.zeros(X.shape[0])

for k in range(self.n_learners):
    yhat += self.alpha[k] * self.learners[k].predict(X)
    scores.append(np.mean(np.sign(yhat) == y))

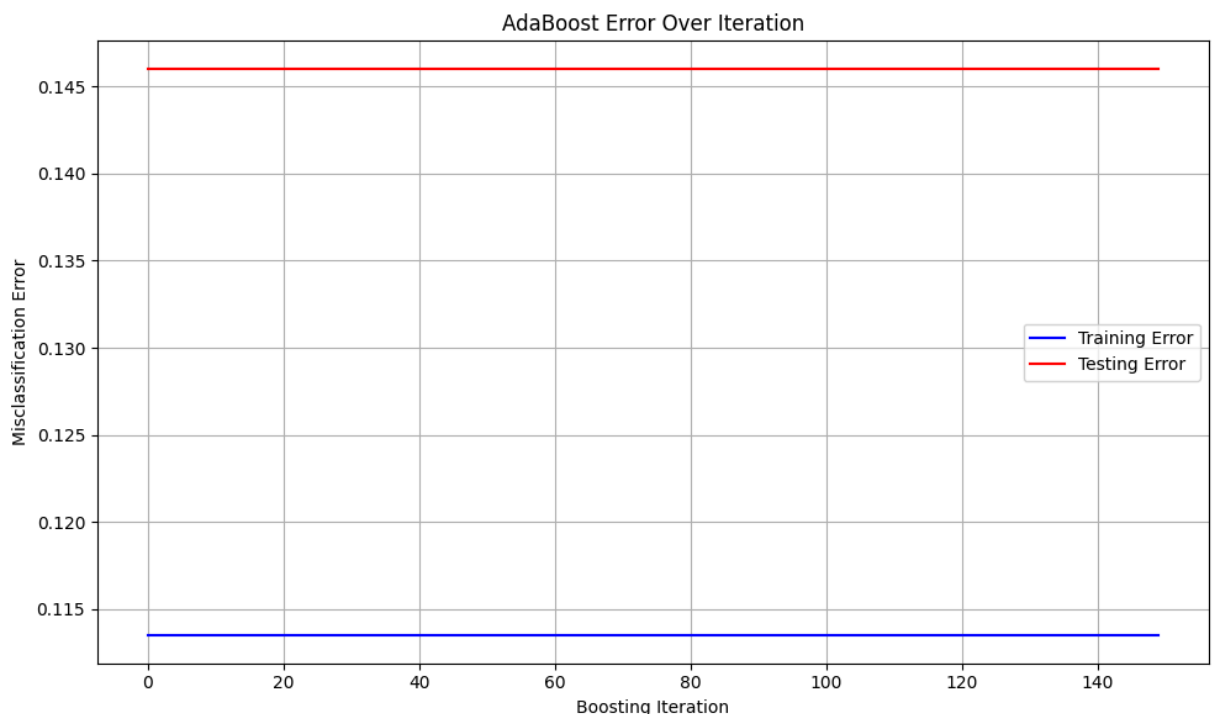
return np.array(scores)

# get accuracy after each boosting round
train_acc = clf.staged_score(data.x_train, data.y_train)
test_acc = clf.staged_score(data.x_test, data.y_test)

# compute misclassification error
train_err = 1 - train_acc
test_err = 1 - test_acc

plt.figure(figsize=(10,6))
plt.plot(train_err, label="Training Error", color='blue')
plt.plot(test_err, label="Testing Error", color='red')
plt.xlabel("Boosting Iteration")
plt.ylabel("Misclassification Error")
plt.title("AdaBoost Error Over Iteration")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

```



Problem 2 [25 pts] : Building an Random Forest Classifier to classify MNIST digits 3 and 8.

Remember that training the random forest algorithms involves the following steps:

for k=1 to K:

a) build kth tree of depth d

b) Return the kth tree trained on the subset of dataset with the random feature splits

Predicting the classification result on new data involves returning the majority vote by all the trees in the random forest.

2.a [10 points]: Complete the `create_tree` function to build a new tree trained on a subset of data. Within this function a decision tree classifier is built and trained on the subset of data with the subset of features.

```
In [29]: class RandomForest():

    def __init__(self, x, y, sample_sz, n_trees=200, n_features='sqrt', max_
        """
        Create a new random forest classifier.

        Args:
            x : Input Feature vector
            y : Corresponding Labels
            sample_sz : Sample size
            n_trees : Number of trees to ensemble
            n_features : Method to select subset of features
            max_depth : Maximum depth of the trees in the ensemble
            min_sample_leaf : Minimum number of samples per leaf
        """
        np.random.seed(12)
        if n_features == 'sqrt':
            self.n_features = int(np.sqrt(x.shape[1]))
        elif n_features == 'log2':
            self.n_features = int(np.log2(x.shape[1]))
        else:
            self.n_features = n_features
        print(self.n_features, "sha: ", x.shape[1])
        self.features_set = []
        self.x, self.y, self.sample_sz, self.max_depth, self.min_samples_leaf
        self.trees = [self.create_tree(i) for i in range(n_trees)]

    def create_tree(self, i):
        """
        create a single decision tree classifier
        """

        idxs = np.random.permutation(len(self.y))[:self.sample_sz]
```



```

idxs = np.asarray(idxs)

f_idx = np.random.permutation(self.x.shape[1])[:self.n_features]
f_idx = np.asarray(f_idx)

if i==0:
    self.features_set = np.array(f_idx, ndmin=2)
else:
    self.features_set = np.append(self.features_set, np.array(f_idx)

# TODO: build a decision tree classifier and train it with x and y t

# your code here

clf = DecisionTreeClassifier(
    max_depth=self.max_depth,
    min_samples_leaf=self.min_samples_leaf
)
x, y = self.x[idxs][:,f_idx], self.y[idxs]

clf.fit(x, y)

return clf

def predict(self, x):

# TODO: create a vector of predictions and return
# You will have to return the predictions of the final ensembles bas

# your code here
# each tree predicts on its own subset of features
preds = np.array([
    tree.predict(x[:, self.features_set[i]])
    for i, tree in enumerate(self.trees)
])

n_samples = preds.shape[1]
y_pred = np.zeros(n_samples, dtype=preds.dtype)

for j in range(n_samples):
    # count occurrences of each unique label among the trees
    values, counts = np.unique(preds[:,j], return_counts=True)

    # majority vote = label with max count
    y_pred[j] = values[np.argmax(counts)]

return y_pred

def score(self, X, y):

# TODO: Compute the score using the predict function and true labels

# your code here

```

```
y_pred = self.predict(X)
return np.mean(y_pred == y)
```

In [30]: *# tests create_tree function*

2.b [15 pts]: In this part you will have to complete three steps:

1. Complete the `predict` function in RandomForest class so as to make predictions using just the features.
2. Finally complete the RandomForest class by completing the `score` function to compute the random forest model's accuracy on any dataset.
3. Build a random forest classifier and train it on the MNIST data to classify 3s and 8s in the cell below. Then see how the classifier performs on the test data by computing the misclassification error. (Remember: error = 1-score)
Answer the Peer Review questions about this section.

In [31]: *# TODO: build a random forest classifier and make predictions*

```
# your code here
rf_clf = RandomForest(
    x=data.x_train,
    y=data.y_train,
    sample_sz=1000,
    n_trees=100,
    n_features='sqrt',
    max_depth=10,
    min_samples_leaf=5
)

# eval test accuracy
test_score = rf_clf.score(data.x_test, data.y_test)

# misclassification error = 1 - accuracy
pred_score = 1 - test_score

print('Misclassification error on test data : %0.3f'%pred_score)
```

21 sha: 441

Misclassification error on test data : 0.050

In []:

In []: